Supervised Dimension Reduction

R. Porcedda, F. Chiaromonte (special thanks to J. Di Iorio, L. Insolia and L. Testa)

March 12th 2026

Introduction

Libraries

We are going to use:

```
library(tidyverse)  # data manipulation and visualization
library(plotly)     # plots in 3D
library(ggplot2)    # plots in 2D
library(ggpubr)     # to combine multiple ggplot objects (ggarrenge)
library(mvtnorm)    # to generate multivariate normal distribution
library(dr)         # SIR
library(factoextra) # PCA-related functions
```

Data

Let's first define a function to generate Gaussian data. This function takes four arguments:

- n: number of observations;
- center: the mean vector
- sigma: the covariance matrix
- label: the cluster label

```
generateGaussianData <- function(n, center, sigma, label) {
  data = rmvnorm(n, center, sigma)
  data = data.frame(data)
  names(data) = c("x", "y", "z")
  data = data %>% mutate(class=factor(label))
  data
}
```

Now let's simulate a dataset.

```
covmat <- diag(3)

# cluster 1
n = 200
center = c(2, 8, 6)
sigma = covmat
group1 = generateGaussianData(n, center, sigma, 1)

# cluster 2
n = 200
center = c(12, 8, 6)
sigma = covmat
group2 = generateGaussianData(n, center, sigma, 2)

# cluster 3
n = 200
center = c(22, 8, 6)
sigma = covmat
group3 = generateGaussianData(n, center, sigma, 3)

# all data
df = bind_rows(group1, group2, group3)

head(df)
```

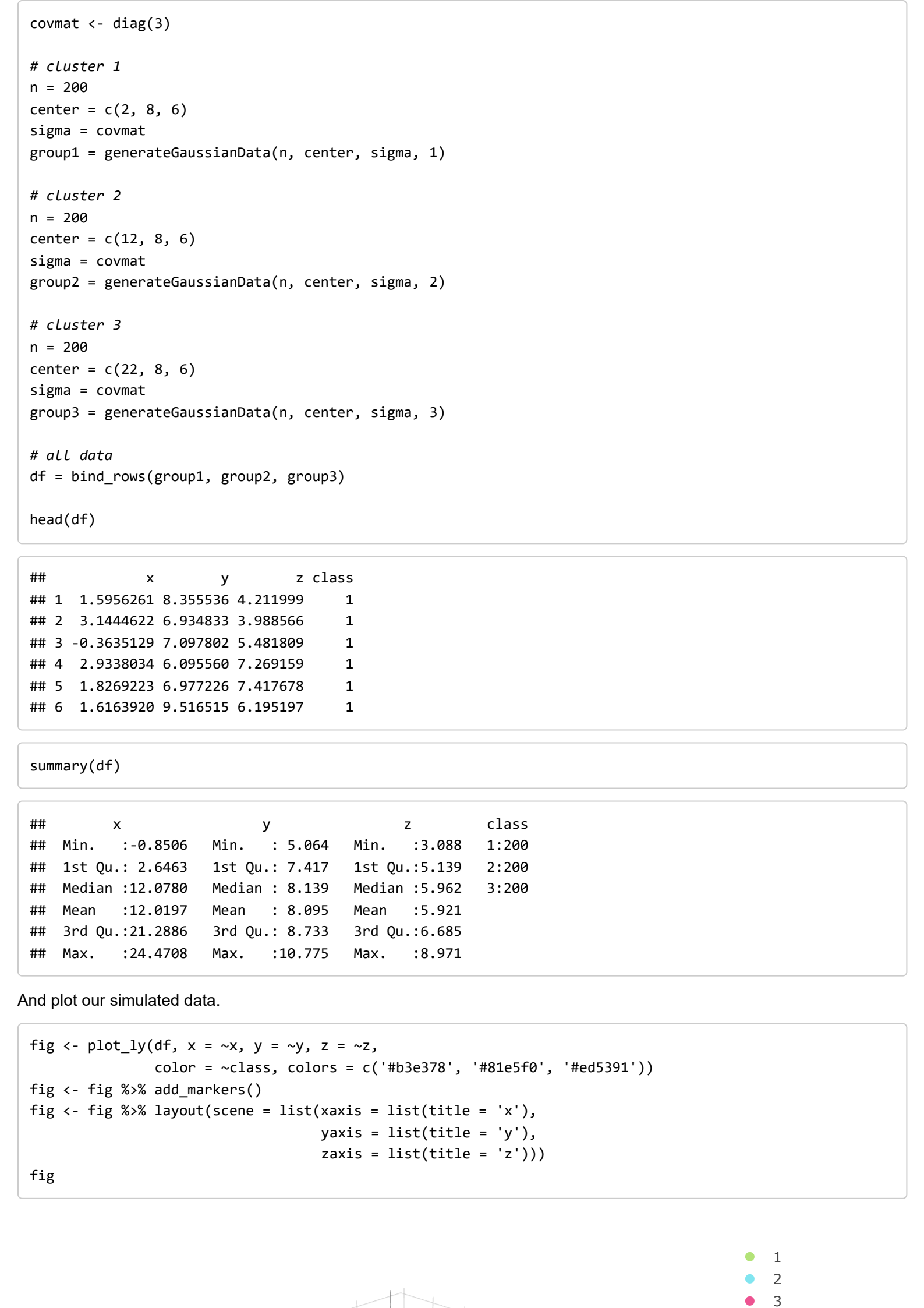
```
##            x        y        z class
## 1  1.5956261 8.355536 4.211999     1
## 2  3.1444622 6.934833 3.988566     1
## 3 -0.3635129 7.097802 5.481809     1
## 4  2.9338034 6.095560 7.269159     1
## 5  1.8269223 6.977226 7.417678     1
## 6  1.6163920 9.516515 6.195197     1
```

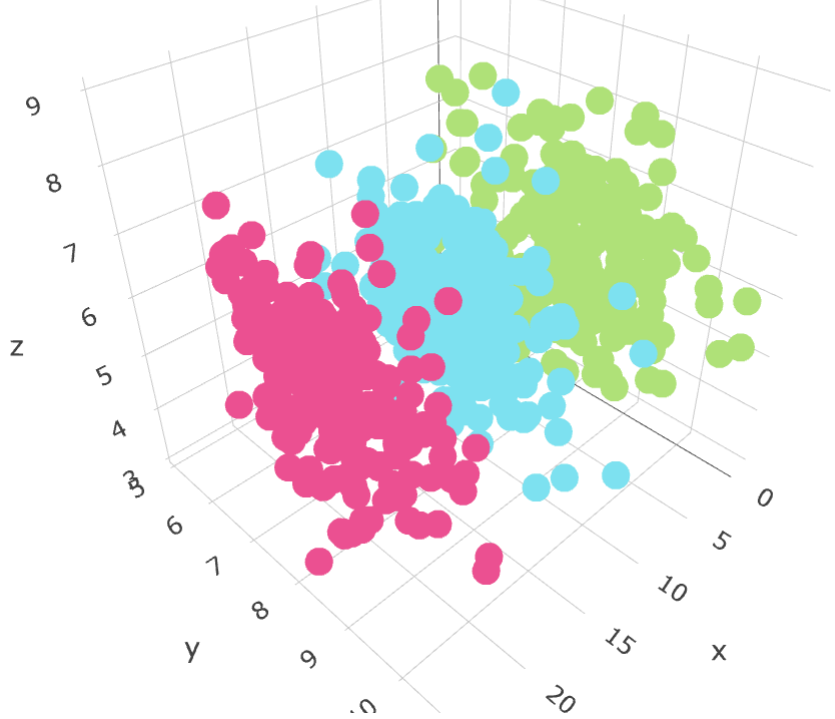
```
summary(df)
```

```
##        x                y                z         class
##  Min.   :-0.8506   Min.   : 5.064   Min.   :3.088   1:200
##  1st Qu.: 2.6463   1st Qu.: 7.417   1st Qu.:5.139   2:200
##  Median :12.0780   Median : 8.139   Median :5.962   3:200
##  Mean   :12.0197   Mean   : 8.095   Mean   :5.921
##  3rd Qu.:21.2886   3rd Qu.: 8.733   3rd Qu.:6.685
##  Max.   :24.4708   Max.   :10.775   Max.   :8.971
```

And plot our simulated data.

```
fig <- plot_ly(df, x = ~x, y = ~y, z = ~z,
               color = ~class, colors = c('#b3e378', '#81e5f0', '#ed5391'))
fig <- fig %>% add_markers()
fig <- fig %>% layout(scene = list(xaxis = list(title = 'x'),
                                   yaxis = list(title = 'y'),
                                   zaxis = list(title = 'z')))
fig
```





PCA vs LDA

PCA

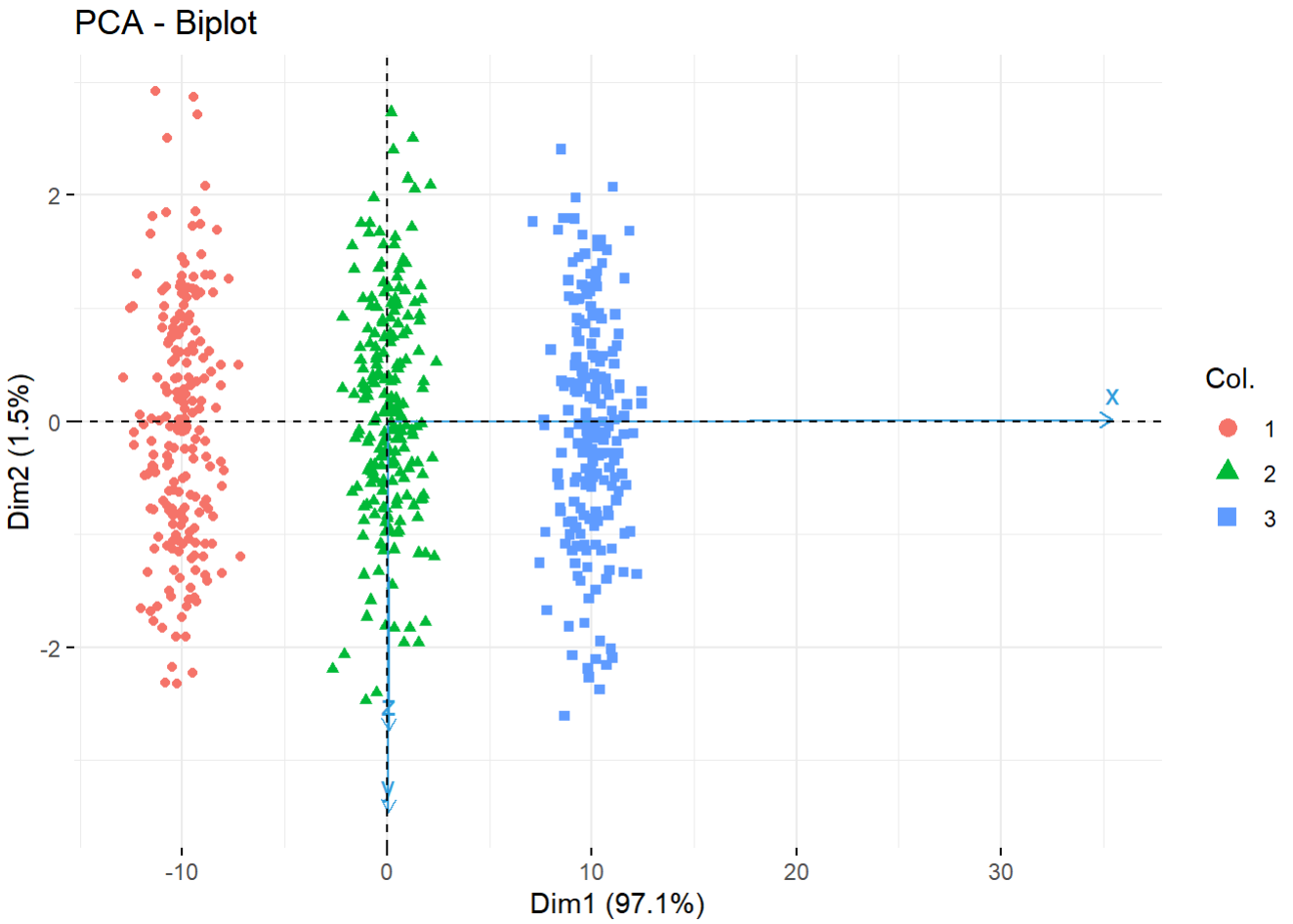
Now let us perform PCA.

```
pc <- prcomp(df[,c(1,2,3)])
get_eig(pc)
```

```
##       eigenvalue variance.percent cumulative.variance.percent
## Dim.1 67.7002505        97.093181                    97.09318
## Dim.2  1.0489279         1.504333                    98.59751
## Dim.3  0.9779124         1.402486                   100.00000
```

This is the corresponding biplot.

```
fviz_pca_biplot(pc, col.var= "#2E9FDF", col.ind= df$class, label="var")
```



Note that considering the first two principal components it is possible to notice differences within the three groups.

LDA

Let's perform LDA:

```
lda.df <- lda(factor(class) ~ x + y + z, data = df)
lda.df
```

```
## Call:
## lda(factor(class) ~ x + y + z, data = df)
##
## Prior probabilities of groups:
##         1         2         3
## 0.3333333 0.3333333 0.3333333
##
## Group means:
##           x        y        z
## 1  1.986197 8.131418 5.898091
## 2 12.097336 7.982861 5.902458
## 3 21.975609 8.171897 5.962168
##
## Coefficients of linear discriminants:
##          LD1          LD2
## x 1.00407954 -0.002338788
## y 0.01114927  0.977727935
## z 0.02607216  0.131983327
##
## Proportion of trace:
##    LD1    LD2
## 0.9999 0.0001
```

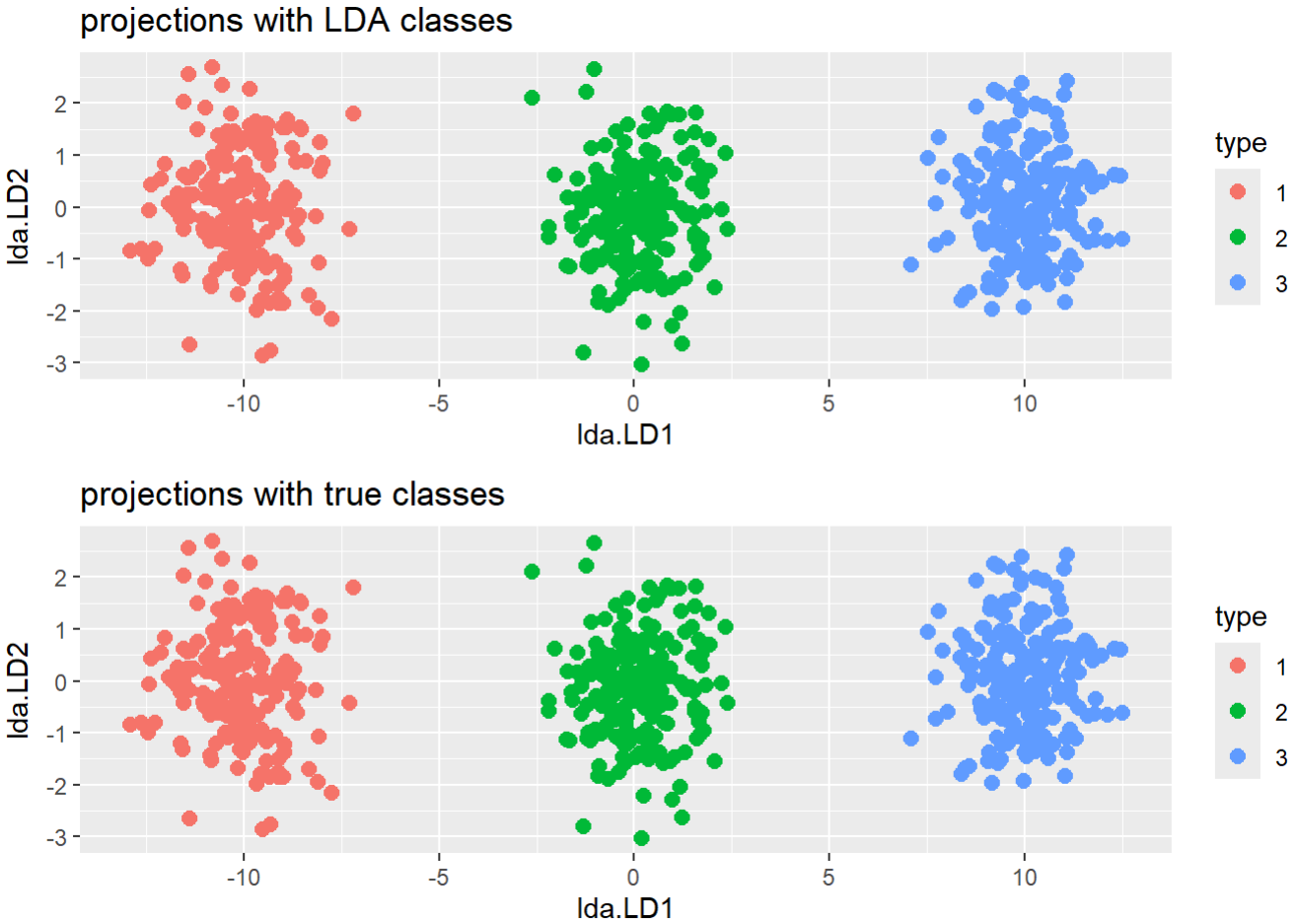
Let us plot the projections on LD1 and LD2

```
# prediction on df to get projections
predmodel.lda = predict(lda.df, data=df)

# projections with LDA classes
estclass <- as.factor(apply(predmodel.lda$posterior, 1, which.max))
newdata2 <- data.frame(type = estclass, lda = predmodel.lda$x)
p1 <- ggplot(newdata2) +
        geom_point(aes(lda.LD1, lda.LD2, colour = type), size = 2.5) +
        ggtitle("projections with LDA classes")

# projections with true classes
newdata <- data.frame(type = df$class, lda = predmodel.lda$x)
p2 <- ggplot(newdata) +
        geom_point(aes(lda.LD1, lda.LD2, colour = type), size = 2.5) +
        ggtitle("projections with true classes")


ggarrange(p1,p2,
          nrow=2)
```



SIR

Now we use the SIR (Sliced Inversion Regression) in the dr package

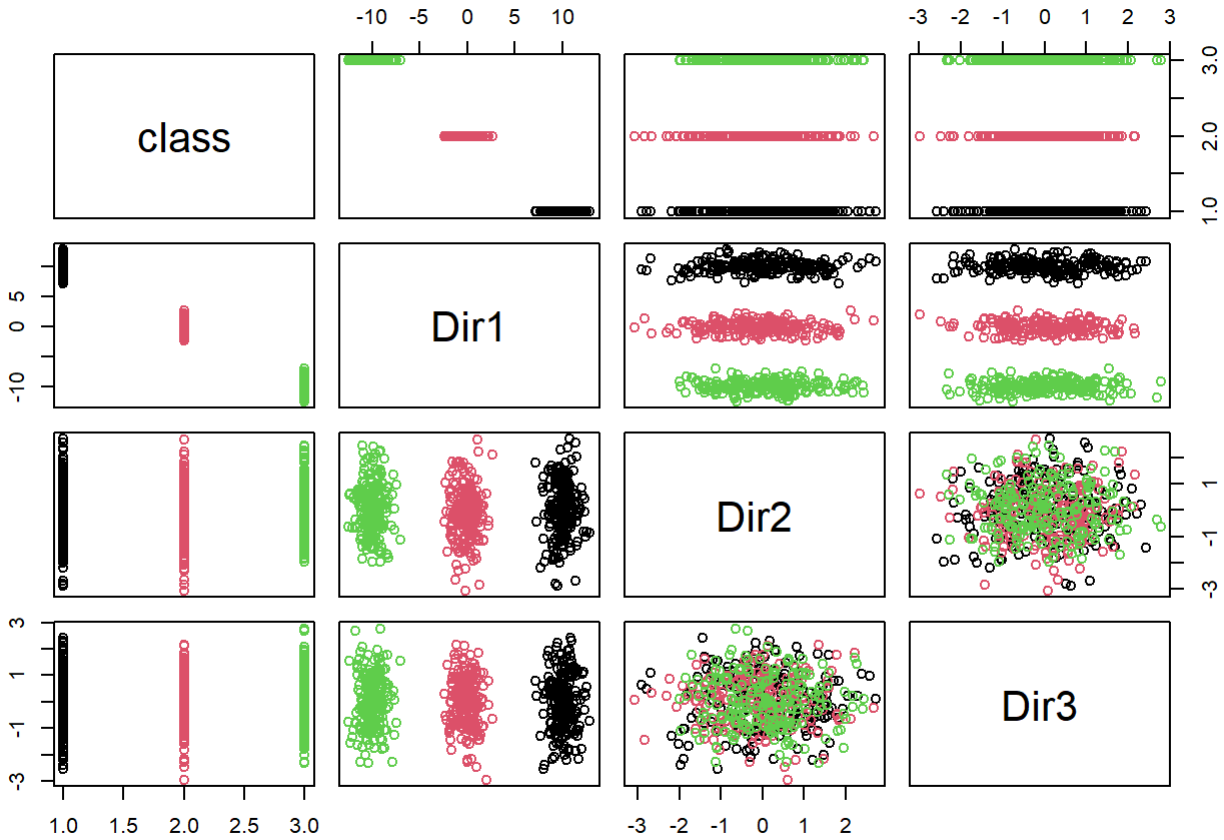
```
# default fitting method is "sir"
help(dr)

dr_res <- dr(class ~ x + y + z, data = df, method='sir')

dr_res
```

```
##
##  dr(formula = class ~ x + y + z, data = df, method = "sir")
## Estimated Basis Vectors for Central Subspace:
##         Dir1         Dir2         Dir3
## x 0.99960147  0.002370556  0.002830867
## y 0.01109955 -0.991008746  0.163678450
## z 0.02595589 -0.133776102 -0.986509681
## Eigenvalues:
## [1] 9.854004e-01 6.335721e-03 7.908748e-19
```

```
plot(dr_res, col=df$class)
```




```
names(dr_res)
```

```
##  [1] "x"           "y"           "weights"     "method"      "cases"
##  [6] "qr"          "group"       "chi2approx"  "evectors"    "evalues"
## [11] "numdir"      "raw.evectors" "M"          "slice.info"  "call"
## [16] "y.name"      "terms"
```

SIR with continuous variable

We generated covariates in a classical continuous framework

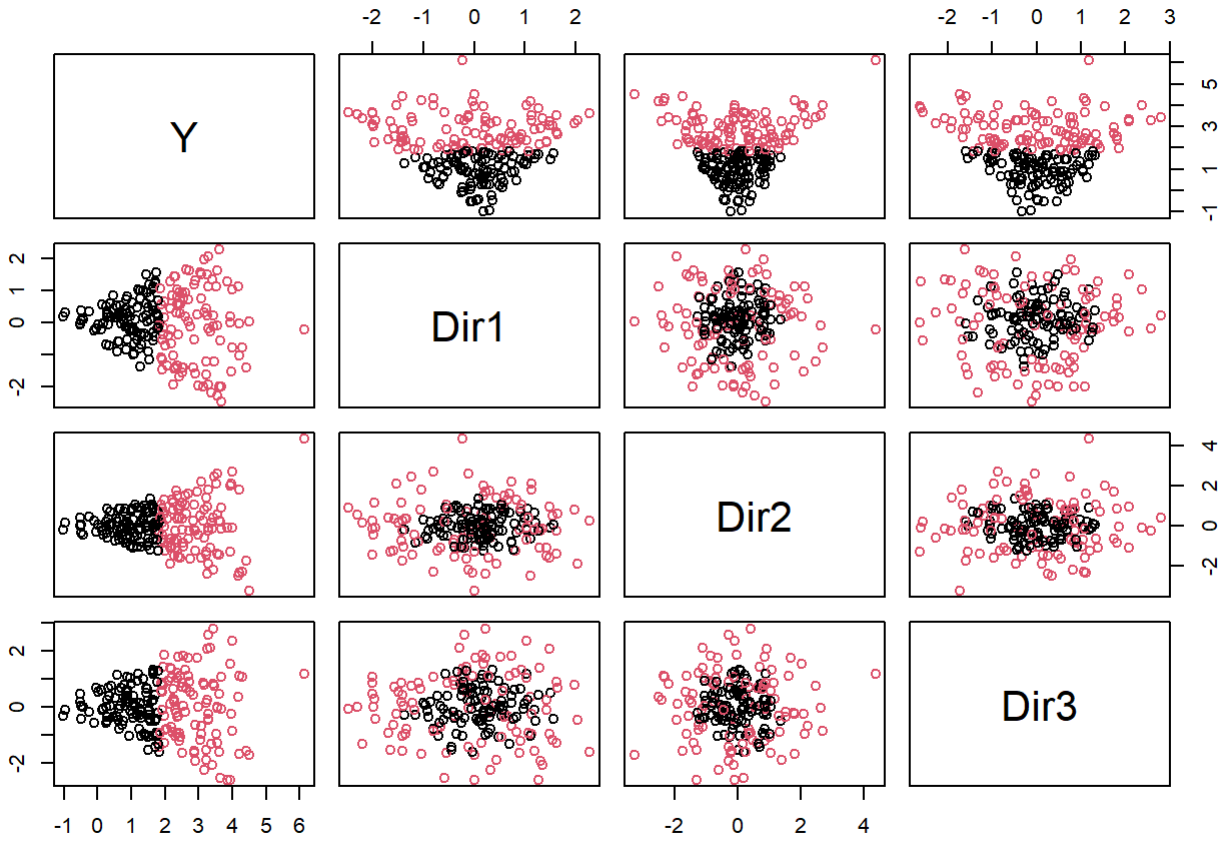
```
set.seed(1)
x <- mvrnorm(n, c(0,0,0), diag(3))
eps <- rnorm(n, 0, 0.3)
Y <- (x[,1]^2 + x[,2]^2 + x[,3]^2)^(1/2) +
  log((x[,1]^2 + x[,2]^2 + x[,3]^2)^(1/2)) + eps
df2 = data.frame(cbind(Y, x))

dr_res2 <- dr(Y ~ ., data = df2, method='sir', nslices=2)

summary(dr_res2)
```

```
##
## Call:
## dr(formula = Y ~ ., data = df2, method = "sir", nslices = 2)
##
## Method:
## sir with 2 slices, n = 200.
##
## Slice Sizes:
## 100 100
##
## Estimated Basis Vectors for Central Subspace:
##      V2      V3      V4
## -0.4236  0.3130  0.8501
##
##                Dir1
## Eigenvalues 0.01179
## R^2(OLS|dr) 0.57983
##
## Large-sample Marginal Dimension Tests:
##              Stat df p.value
## 0D vs >= 1D 2.357  3  0.5017
```

```
plot(dr_res2, col=dr_res2$slice.info$slice.indicator)
```



Now we perform the same analysis, but with 5 slices

```
Y <- (x[,1]^2 + x[,2]^2 + x[,3]^2)^(1/2) +
  log((x[,1]^2 + x[,2]^2 + x[,3]^2)^(1/2)) + eps
df2 = data.frame(cbind(Y, x))

dr_res2 <- dr(Y ~ ., data = df2, method='sir', nslices=5)

summary(dr_res2)
```

```
##
## Call:
## dr(formula = Y ~ ., data = df2, method = "sir", nslices = 5)
##
## Method:
## sir with 5 slices, n = 200.
##
## Slice Sizes:
## 40 40 40 40 40
##
## Estimated Basis Vectors for Central Subspace:
##       Dir1     Dir2     Dir3
## V2  0.3836 -0.89997 -0.04448
## V3  0.2032  0.03049  0.96656
## V4 -0.9009 -0.43489  0.25255
##
##                 Dir1     Dir2     Dir3
## Eigenvalues 0.01852 0.004285 4.86e-05
## R^2(OLS|dr) 0.87454 0.999623 1.00e+00
##
## Large-sample Marginal Dimension Tests:
##                  Stat df p.value
## 0D vs >= 1D 4.571726 12  0.9708
## 1D vs >= 2D 0.866749  6  0.9902
## 2D vs >= 3D 0.009721  2  0.9952
```

```
plot(dr_res2, col=dr_res2$slice.info$slice.indicator)
```